Practical Preimages for 19- and 20-Step Reduced SHA-256: Exact Algebraic Padding Inversion, Context Shaping, and the Carry/Slot-Sudoku Framework

Author: Sebastian Wain **Date:** October 2026 **Companion Pedagogical Document:** *Carry-Sudoku & Slot-Sudoku in Reduced SHA-256: A Self-Contained ELI5 and ELI-Undergraduate Companion Guide* (`COMPANION_SUDOKU_ELI5_ELIU.pdf`) **Repositories & Artifacts:**

- 19-Step & 20-Step Padded Witnesses, Paper & Solvers: `https://github.com/srw/sha256-19step-preimages-with-padding`
- 20-Step Unpadded Compression Solver & Witness: `https://github.com/srw/sha256-20step-preimages-with-no-padding`
- Local Solver & Verification Suite: `solve20_padded_fast.c` (CPU), `solve20_padded_cuda.cu` (CUDA GPU), `solve20_zero_a3.c` (Zero-a_3 Unpadded), `FOUND_20ROUND_PREIMAGE.txt` , `verify_all_witnesses.py`

Abstract

We present the first demonstrated preimages of both the **19-step** and **20-step SHA-256 hash functions**: standard 55-byte single-block (and 119-byte two-block) messages with the standard initial value (IV), standard Davies–Meyer feed-forward, and full FIPS 180-4 Merkle–Damgård padding (`0x80` terminator byte, zero fill, and 64-bit big-endian bit length $L = 440$) whose 19-step or 20-step digest equals a prescribed 256-bit target. Earlier computed preimages at 18–20 steps inverted only the unpadded compression function (16 unconstrained 32-bit message words), while padding-aware meet-in-the-middle and biclique attacks at 24–47 steps remain strictly theoretical ($\geq 2^{240}$ compression calls).

Our solver builds upon K. S. Basra's selector-saturation framework ($e_9 = e_{10} =$ `0xffffffff`) and introduces a unified algebraic and data-structural methodology—which we term **Carry-Sudoku** and **Slot-Sudoku** (see the companion pedagogical guide `COMPANION_SUDOKU_ELI5_ELIU.pdf` for intuitive ELI5 and undergraduate derivations of every equation `[E1]` – `[E41]`):

1. **Exact $O(1)$ Algebraic Inversion of FIPS 180-4 Padding (`[E15]` – `[E22]`):** Exploiting the bit-disjointness of $\Sigma_1(e_{13}) \bmod 256$ with respect to bits $18..31$ of e_{13}, we invert the padding byte constraint $(W_{13} \bmod 256) =$ `0x80` in $O(1)$ time with 100% branchless inner-loop validity ($256\times$ faster than filtering). Combined with algebraic Ch-annihilation ($e_{15} =$ `0xffffffff`, $a_{10} = a_{11}$), this eliminates "soft shaping" ($\mathrm{popcount}(a_5 \oplus a_6) \leq 3$) and produces **exact** $D = a_5 \oplus a_6 = 0$ **padded contexts**.
2. **Carry-Sudoku: 7-Stage Causal Bit-Slice & Rotation-Invariant Right-Context Solver (`[E23]` – `[E26]`):** We prove the **Exact Equivalence Theorem** (`[E23]`): $a_5 = a_6$ is equivalent to two bit-orthogonal conditions on $S = X + \mathrm{Ch}(D, X, a_8)$ where $D = a_8 \oplus a_7$: a lower-triangular carry condition $((S \oplus X) \wedge D) = 0$ that is **100% independent of a_6 and $\Sigma_0(a_9)$**, and a complementary condition $((a_6 \oplus S) \wedge \neg D) = 0$. Because modular addition propagates carries strictly from LSB to MSB, we prune candidate bits of e_{13} across 7 causal bit-slices (`4, 8, 12, 15, 19, 22, 32` bits) plus rotation-invariant windows of $\Sigma_0(a_9)$ (`bits 13..15` and `3..4`), achieving 86.4 billion trials/s on a 12-core Apple M3 CPU (`0.11 s` per exact $D = 0$ context) and 56 trillion trials/s (`29,310` exact $D = 0$ contexts/s) on a single NVIDIA L4 GPU.
3. **Slot-Sudoku & Zero-Table Closed-Form $\mathbb{F}_2$ Inverse (`[E27]` – `[E37]`):** We prove that given $y = Q_0(u) = \sigma_0(u) - u$ and the lower 16 bits $u_{\mathrm{lo}} = u \bmod 2^{16}$, the upper 16 bits u_{hi} satisfy a 3-step triangular $\mathbb{F}_2$ recurrence with an exact **5-instruction closed-form bitwise inverse** (`[E29]`), eliminating the 128 KB reconstruction table (~200 billion random L1/L2 loads per sweep). To invert Q_0 across three chained steps ($a_1 \to a_2 \to a_3 \to a_4$), we introduce **Slot-Sudoku**: a **6.00 GB 2-Tier Nested 4-Slot Earliest-Deadline-First (EDF) + Cuckoo-Sudoku table** capturing 82.98% **of all 2^{32} preimages** (57.13% 3-level chain survival) in laptop/GPU memory, alongside a **16.00 GB Direct-Indexed 2-Way Exact GPU VRAM table** (79.64% single-level, 50.51% 3-level exact survival with zero false-bucket checks) and a **32.00 GB 4-Way Exact CPU table** (99.57% coverage, 98.72% 3-level survival).
4. **First Demonstrated 20-Step Padded SHA-256 Preimage & 180.8k `sol19/s` GPU Engine (§8–§9):** Every candidate surviving Level 3 is a complete **19-step padded preimage**, and a single terminal equation $\sigma_0(W_4) + W_3 \equiv T_{19} \pmod{2^{32}}$ (`[E36]` ,

with exact $\mathbb{F}_2^{32}$ bijection $W_4 = \sigma_0^{-1}(T_{19} - W_3)$, `[E37]`) filters that stream directly for **20-step padded preimages**. On an 8× NVIDIA L4 Spot GPU cluster, our solver discovered the **first demonstrated 20-step padded SHA-256 preimage** (for $H_{20}^* =$ `0000...0000`, 55-byte message `ec531306...65416f`) after $70{,}661{,}112$ exact 19-step padded solutions (~19 minutes, < \$0.90 Spot cost). With Multi-Way Single-T_{16} Fusion ($B = 16$) and Warp-Compacted Shared-Memory Level-3 Queues, single-GPU throughput reaches $180{,}831.9\ \text{sol19/s}$ on 1× NVIDIA L4 ($3.36\times$ over baseline), while our Apple M3 CPU solver reaches $6{,}242.8\ \text{sol19/s}$ (reducing single-M3 expected 20-step runtime from 7.6 days to 11.9 hours).

Finally, for the unpadded 20-step compression function (§7), we present the **Zero-a_3 Bi-Directional Q_0/Q_1 Bridge** (`[E38]`–`[E41]`), which annihilates both majority nonlinearities for 100% of $a_4 \in [0, 2^{32})$ and finds a verified 20-step compression preimage for `deadbeefcafebabe...` in **21.1 minutes** on 16 CPU cores. These results concern step-reduced SHA-256 (19–20 of 64 steps) and do not threaten full 64-step SHA-256.

---

1. Introduction

SHA-256 [FIPS 180-4] hashes an L-bit message M by appending mandatory Merkle–Damgård strengthening (FIPS 180-4 §5.1.1)—a single `'1'` bit (`0x80` at a byte boundary), zero padding bits, and the 64-bit big-endian bit length L in the final two 32-bit words W_{14} and W_{15}—before applying a 64-step Davies–Meyer compression function initialized to the standard initial value IV.

In the cryptanalysis of step-reduced SHA-256, a persistent divide has separated **theoretical** attacks from **practical (computed)** preimages:

- **Theoretical preimage attacks** (splice-and-cut meet-in-the-middle, partial matching, bicliques, and dual-syncopation MitM) respect FIPS 180-4 padding from 24 steps [Isobe–Shibutani, FSE 2009] through 41–43 steps [Sasaki et al. 2009; Aoki et al., ASIACRYPT 2009], 45 steps [Khovratovich–Rechberger–Savelieva, FSE 2012], and 46–47 steps [Guo et al., EUROCRYPT 2026], but require between 2^{240} and $2^{255.6}$ compression calls and have never been executed to produce a preimage.
- **Practical (demonstrated) preimages** via algebraic/SAT encodings reached 16 steps with padding [Homsirikamol et al., CISIM 2012] and 3 unverified mailing-list messages at 17 steps [McDair 2024], whereas SAT/Cube-and-Conquer [Davydov et al. 2025; Zaikin, *Constraints* 2026] and algebraic context shaping [Basra 2026] reached 18, 19, and 20 steps **exclusively for the unpadded compression function**, where all 16 message words $W_0, \ldots, W_{15}$ are unconstrained 32-bit degrees of freedom.

For a single-block 55-byte ($L = 440$-bit) padded message, FIPS 180-4 fixes 72 bits at the end of the message block:

$$W_{15} = 440\ (\texttt{0x000001b8}), \qquad W_{14} = 0\ (\texttt{0x00000000}), \qquad W_{13} \bmod 256 = \texttt{0x80}. \tag{E1}$$

In backward-and-forward inversion of steps $0..18$ (19-step) or $0..19$ (20-step), W_{13}, W_{14}, W_{15} are tightly coupled with the message schedule expansion $W_{16}, \ldots, W_{19}$ and the internal state window $(a_5, \ldots, a_{12})$. Satisfying all 72 padding bits simultaneously with the selector-saturation and majority-absorption conditions that decouple the left and right halves of the compression function is the central barrier between unpadded compression preimages and true padded hash preimages.

> **Reader's Guide & Companion Pedagogical Document:**
>
> Every equation tag **`[E1]`–`[E41]`** in this paper is cross-referenced in our standalone pedagogical companion document, **`COMPANION_SUDOKU_ELI5_ELIU.pdf`** (*"Carry-Sudoku & Slot-Sudoku in Reduced SHA-256: A Self-Contained ELI5 and ELI-Undergraduate Companion Guide"*), which provides a self-contained glossary, intuitive **ELI5 (Explain Like I'm 5)** analogies, and **ELI-Undergraduate** step-by-step derivations linked directly to the C and CUDA source code.

---

2. Preliminaries and the Scalar a-Sequence Formulation

All arithmetic is modulo 2^{32}. Denote 32-bit right rotation by $\mathrm{ROR}(x, n)$ and logical right shift by $\mathrm{SHR}(x, n)$. The SHA-256 boolean and $\mathbb{F}_2$-linear functions are:

$$\begin{aligned} \mathrm{Ch}(e,f,g) &= (e \wedge f) \oplus (\neg e \wedge g), & \mathrm{Maj}(a,b,c) &= (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c), \\ \Sigma_0(x) &= \mathrm{ROR}(x,2) \oplus \mathrm{ROR}(x,13) \oplus \mathrm{ROR}(x,22), & \Sigma_1(x) &= \mathrm{ROR}(x,6) \oplus \mathrm{ROR}(x,11) \oplus \mathrm{ROR}(x,25), \\ \sigma_0(x) &= \mathrm{ROR}(x,7) \oplus \mathrm{ROR}(x,18) \oplus \mathrm{SHR}(x,3), & \sigma_1(x) &= \mathrm{ROR}(x,17) \oplus \mathrm{ROR}(x,19) \oplus \mathrm{SHR}(x,10). \end{aligned} \tag{E2}$$

For step indices $t \geq 16$, the message schedule expands $W_0, \ldots, W_{15}$ via:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}. \tag{E3}$$

We define the coupled schedule operator $Q_0 : \mathbb{Z}_{2^{32}} \to \mathbb{Z}_{2^{32}}$ by:

$$Q_0(u) = \sigma_0(u) - u \pmod{2^{32}}. \tag{E4}$$

2.1 Reduction to the Scalar a-Sequence (`[E5]` – `[E7]`)

Let $(a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t)$ be the 8 working registers after step $t-1$ (with $t=0$ denoting IV). Because each step shifts $b \leftarrow a, c \leftarrow b, d \leftarrow c$ and $f \leftarrow e, g \leftarrow f, h \leftarrow g$, the entire 8-register trajectory across R steps is uniquely parameterized by a single scalar sequence $a_{-7}, a_{-6}, \ldots, a_R$ (see Companion Guide §1):

- **Initial Boundary ($s \in \{-7, \ldots, 0\}$):** Fixed uniquely by IV (`window_from_state`):

$$(a_0, a_{-1}, a_{-2}, a_{-3}) = (\mathrm{IV}_0, \mathrm{IV}_1, \mathrm{IV}_2, \mathrm{IV}_3), \qquad (e_0, e_{-1}, e_{-2}, e_{-3}) = (\mathrm{IV}_4, \mathrm{IV}_5, \mathrm{IV}_6, \mathrm{IV}_7). \tag{E5}$$

- **Terminal Boundary ($s \in \{R-7, \ldots, R\}$):** For an R-step target digest $H^* = (H_0^*, \ldots, H_7^*)$, Davies–Meyer feed-forward fixes the terminal state $st_i = H_i^* - \mathrm{IV}_i$, which uniquely determines the 8 terminal words $a_{R-7}, \ldots, a_R$:

$$\begin{aligned} &\text{For } R = 19: \quad a_{12}, a_{13}, \ldots, a_{19} \text{ are fixed by } H^* - \mathrm{IV}, \\ &\text{For } R = 20: \quad a_{13}, a_{14}, \ldots, a_{20} \text{ are fixed by } H^* - \mathrm{IV}. \end{aligned} \tag{E6}$$

- **Internal State & Message Recovery:** For every step index s and $t \in \{0, \ldots, R-1\}$, the e-register e_s and message word W_t are exact functions of five consecutive a-words (`E_arr` and `Wt_arr`):

$$\begin{aligned} e_s &= a_s - \Sigma_0(a_{s-1}) - \mathrm{Maj}(a_{s-1}, a_{s-2}, a_{s-3}) + a_{s-4}, \\ W_t &= e_{t+1} - a_{t-3} - e_{t-3} - \Sigma_1(e_t) - \mathrm{Ch}(e_t, e_{t-1}, e_{t-2}) - K_t. \end{aligned} \tag{E7}$$

Thus, for $R = 20$ (resp. $R = 19$), the only free internal state variables between the IV boundary and the target boundary are the **12 words** $a_1, a_2, \ldots, a_{12}$ (resp. **11 words** $a_1, \ldots, a_{11}$). Any assignment to these words automatically hashes to H^* after R steps if and only if the recovered message words $W_0, \ldots, W_{R-1}$ from `[E7]` satisfy the message schedule equations `[E3]` for $t = 16, \ldots, R-1$ and the FIPS 180-4 padding constraints `[E1]` .

---

3. Selector Saturation and Exact Context Shaping ($D = 0$)

In general, $W_{16}, W_{17}, W_{18}, W_{19}$ depend on the Left variables (a_1, a_2, a_3, a_4) both through the early message words $(W_0, W_1, W_2, W_3, W_4)$ on the right-hand side of `[E3]` and through $(W_9, W_{10}, W_{11}, W_{12}, W_{14}, W_{15})$ via the state words e_5, e_6, e_7, e_8.

Following Basra's selector-saturation principle, we pin the two middle selector registers:

$$e_9 = \texttt{0xffffffff}, \qquad e_{10} = \texttt{0xffffffff}. \tag{E8}$$

Because $\mathrm{Ch}(\texttt{0xffffffff}, y, z) = y$, setting $e_9 = e_{10} = \texttt{0xffffffff}$ blocks (e_7, e_8) inside $\mathrm{Ch}(e_9, e_8, e_7) = e_8$, $\mathrm{Ch}(e_{10}, e_9, e_8) = \texttt{0xffffffff}$, and $\mathrm{Ch}(e_{11}, e_{10}, e_9) = \texttt{0xffffffff}$. Expanding `[E7]` at $s = 9, 10$ expresses a_5 and a_6 purely in terms of the Right words (a_7, a_8, a_9, a_{10}):

$$
\begin{aligned}
a_5 &= \texttt{0xffffffff} - a_9 + \Sigma_0(a_8) + \mathrm{Maj}(a_8, a_7, a_6), \\
a_6 &= \texttt{0xffffffff} - a_{10} + \Sigma_0(a_9) + \mathrm{Maj}(a_9, a_8, a_7).
\end{aligned}
\tag{E9}
$$

3.1 Exact Shaping ($D = a_5 \oplus a_6 = 0$) (`[E10]` – `[E14]`)

Let $D_{\text{shape}} = a_5 \oplus a_6$. Under `[E8]`, the dependence of $W_9, W_{10}, W_{11}, W_{12}$ on the Left variables (a_1, a_2, a_3, a_4) collapses to:

$$
\begin{aligned}
W_9 &= c_9 - a_1, \\
W_{10} &= c_{10} - a_2, \\
W_{11} &= c_{11} - a_3 - \big(\mathrm{Maj}(a_6, a_5, a_4) - (a_6 \wedge a_5)\big), \\
W_{12} &= \mathrm{const}(\mathrm{Right}) + a_4 - \mathrm{Maj}(a_5, a_4, a_3),
\end{aligned}
\tag{E10}
$$

where c_9, c_{10}, c_{11} depend **only** on the Right context $(a_5, \ldots, a_{12})$.

When we enforce **exact context shaping**:

$$
D_{\text{shape}} = a_5 \oplus a_6 = 0 \iff a_5 = a_6,
\tag{E11}
$$

the majority term in W_{11} becomes identically constant for **every** $a_4 \in \mathbb{Z}_{2^{32}}$:

$$
\mathrm{Maj}(a_6, a_5, a_4) = \mathrm{Maj}(a_5, a_5, a_4) = a_5 = a_6 \wedge a_5 \implies W_{11} = c_{11} - a_3 \quad (\text{exact, zero } a_4 \text{ leakage!}).
\tag{E12}
$$

(In our first-generation 19-step solver `src/pre19.c`, we used "soft shaping" $\mathrm{popcount}(a_5 \oplus a_6) \le 3$*, which leaked* a_4 *into* W_{11} *on up to 3 bits and reduced chain survival by* $2^{\mathrm{popcount}(a_5 \oplus a_6)} = 2 \times -8\times$*. Exact shaping `[E11]` eliminates this penalty completely.)*

Simultaneously, the remaining nonlinear condition in W_{12} (`[E10]`):

$$
\mathrm{Maj}(a_5, a_4, a_3) = a_4 \iff \big((a_4 \oplus a_5) \wedge M_3\big) = 0, \qquad M_3 := \neg(a_3 \oplus a_5),
\tag{E13}
$$

causes $+a_4$ inside e_8 and $-\mathrm{Maj}(a_5, a_4, a_3) = -a_4$ inside e_7 to cancel identically in W_{12}. Consequently, whenever `[E11]` and `[E13]` hold, $W_{12}, W_{13}, \ldots, W_{19}$ are **100% independent of** (a_1, a_2, a_3, a_4).

Substituting $W_0 = a_1 + C_{W_0}$ and $a_{t+1} = W_t + G_t(a_1, \ldots, a_t)$ into the message schedule equations `[E3]` for $t = 16, 17, 18$ rewrites them as three chained Q_0-inversions:

$$
\begin{aligned}
&\text{Level 1 } (t=16): && Q_0(W_1) = T_{16} + F_1(a_1), && T_{16} := W_{16} - \sigma_1(W_{14}) - c_9, \\
&\text{Level 2 } (t=17): && Q_0(W_2) = T_{17} + G_2(a_1, a_2) - W_1, && T_{17} := W_{17} - \sigma_1(W_{15}) - c_{10}, \\
&\text{Level 3 } (t=18): && Q_0(W_3) = T_{18} + G_3(a_1, a_2, a_3) - W_2, && T_{18} := W_{18} - \sigma_1(W_{16}) - c_{11}.
\end{aligned}
\tag{E14}
$$

---

4. Exact $O(1)$ Algebraic Inversion of FIPS 180-4 Padding (`W13..W15`)

To construct a valid 55-byte single-block padded message (`[E1]`), the Right variables $(a_5, \ldots, a_{12})$ must simultaneously satisfy $W_{15} = 440$, $W_{14} = 0$, $(W_{13} \bmod 256) = \texttt{0x80}$, $e_9 = e_{10} = \texttt{0xffffffff}$ (`[E8]`), and $a_5 = a_6$ (`[E11]`).

4.1 Closed-Form Absorption of $W_{15} = 440$ and $W_{14} = 0$ (`[E15]` – `[E17]`)

Given (a_{10}, a_{11}, a_{12}) (for $R = 20$, where $a_{13..20}$ are fixed by H^*), the registers e_{16}, e_{15}, e_{14} are determined via `[E7]` :

$$
\begin{aligned}
e_{16} &= a_{16} - \Sigma_0(a_{15}) - \mathrm{Maj}(a_{15}, a_{14}, a_{13}) + a_{12}, \\
e_{15} &= a_{15} - \Sigma_0(a_{14}) - \mathrm{Maj}(a_{14}, a_{13}, a_{12}) + a_{11}, \\
e_{14} &= a_{14} - \Sigma_0(a_{13}) - \mathrm{Maj}(a_{13}, a_{12}, a_{11}) + a_{10}.
\end{aligned}
\tag{E15}
$$

Expanding $W_{15} = 440$ and $W_{14} = 0$ via `[E7]` isolates e_{12} and e_{11} as closed-form functions of a single 32-bit word e_{13} (equivalently $a_9 = e_{13} - c_{e_{13}}$):

$$\begin{aligned} e_{12} &= \underbrace{(e_{16} - a_{12} - \Sigma_1(e_{15}) - K_{15} - 440)}_{c_{e_{12}}} - \mathrm{Ch}(e_{15}, e_{14}, e_{13}), \\ e_{11} &= \underbrace{(e_{15} - a_{11} - \Sigma_1(e_{14}) - K_{14})}_{c_{e_{11}}} - \mathrm{Ch}(e_{14}, e_{13}, e_{12}). \end{aligned} \tag{E16}$$

Then a_8 and a_7 are recovered in closed form:

$$a_8 = e_{12} + \Sigma_0(a_{11}) - a_{12} + \mathrm{Maj}(a_{11}, a_{10}, a_9), \qquad a_7 = e_{11} + \Sigma_0(a_{10}) - a_{11} + \mathrm{Maj}(a_{10}, a_9, a_8). \tag{E17}$$

Thus **every** choice of $(a_{12}, a_{11}, a_{10}, e_{13})$ satisfies $W_{15} = 440$ and $W_{14} = 0$ identically.

4.2 Exact $O(1)$ Pivot Inversion of $(W_{13} \bmod 256) = \texttt{0x80}$ (`[E18]` – `[E22]`)

Since $e_{10} = \texttt{0xffffffff}$, the formula for W_{13} (`[E7]`) is:

$$W_{13} = \underbrace{(e_{14} - a_{10} - \texttt{0xffffffff} - K_{13})}_{c_{W_{13}}} - \Sigma_1(e_{13}) - \mathrm{Ch}(e_{13}, e_{12}, e_{11}). \tag{E18}$$

Reducing `[E18]` modulo 256:

1. Because Ch is bitwise and modular subtraction propagates carries only from lower bits to higher bits, the low 8 bits $e_{12,\mathrm{lo}} := e_{12} \bmod 256$ and $e_{11,\mathrm{lo}} := e_{11} \bmod 256$ in `[E16]` depend **only on the low 8 bits** $e_{13,\mathrm{lo}} := e_{13} \bmod 256$.
2. Once $e_{13,\mathrm{lo}} \in [0, 255]$ is chosen, the required low byte of $\Sigma_1(e_{13})$ is fixed:

$$R_{\mathrm{lo}} := \big(c_{W_{13}} - \mathrm{Ch}(e_{13,\mathrm{lo}}, e_{12,\mathrm{lo}}, e_{11,\mathrm{lo}}) - \texttt{0x80}\big) \bmod 256. \tag{E19}$$

3. Inspecting bit $b \in \{0, \ldots, 7\}$ of $\Sigma_1(e_{13}) = \mathrm{ROR}(e_{13}, 6) \oplus \mathrm{ROR}(e_{13}, 11) \oplus \mathrm{ROR}(e_{13}, 25)$:

$$\big(\Sigma_1(e_{13})\big)_b = (e_{13})_{b+6} \oplus (e_{13})_{b+11} \oplus (e_{13})_{(b+25) \bmod 32}. \tag{E20}$$

- For $b \in \{0, \ldots, 6\}$, $(b + 25) \bmod 32 = b + 25 \in \{25, \ldots, 31\}$, which appears **nowhere else** in `[E20]` !
- For $b = 7$, $(7 + 25) \bmod 32 = 0 \in \{0, \ldots, 7\}$, while $(e_{13})_{7+11} = (e_{13})_{18}$ appears **nowhere else** in `[E20]` !
- Furthermore, bits $19..24$ of e_{13} **do not appear anywhere in** `[E20]` and **do not affect** $e_{13,\mathrm{lo}}$!

Therefore, for any assignment to bits $0..17$ of e_{13} ($e_{13,\mathrm{base}}$), the 8 pivot bits (`bit 18` and `bits 25..31`) are uniquely determined in closed form:

$$\Delta := \big(R_{\mathrm{lo}} \oplus \Sigma_1(e_{13,\mathrm{base}})\big) \bmod 256, \qquad e_{13,\mathrm{fixed}} := e_{13,\mathrm{base}} \mid \big((\Delta \wedge \texttt{0x7f}) \ll 25\big) \mid \big(((\Delta \gg 7) \wedge 1) \ll 18\big), \tag{E21}$$

while bits $19..24$ remain 6 completely free degrees of freedom where 100% **of assignments satisfy** $(W_{13} \bmod 256) = \texttt{0x80}$.

Finally, in `solve20_padded_fast.c` and `solve20_padded_cuda.cu` , we pin $e_{15} = \texttt{0xffffffff}$ (fixing a_{11} from a_{12} via `[E15]`) and set $a_{10} = a_{11}$. This simultaneously forces $\mathrm{Ch}(e_{15}, e_{14}, e_{13}) = e_{14}$ (**making e_{12} and a_8 100% constant across all 2^{24} choices of e_{13}!**), simplifies $\mathrm{Maj}(a_{11}, a_{10}, a_9) = a_{11}$, and linearizes $\mathrm{Maj}(a_{10}, a_9, a_8) = (a_{10} \wedge a_8) + ((a_{10} \oplus a_8) \wedge a_9)$:

$$e_{15} = \texttt{0xffffffff}, \quad a_{10} = a_{11} \implies e_{12} = c_{e_{12}} - e_{14} = \mathrm{const}, \quad a_8 = e_{12} + \Sigma_0(a_{11}) - a_{12} + a_{11} = \mathrm{const}. \tag{E22}$$

---

5. Carry-Sudoku: 7-Stage Causal Bit-Slice & Rotation-Invariant Right-Context Search

Even with `[E21]` – `[E22]`, each a_{12} defines 2^{24} padded candidates for e_{13}, and testing $a_5 = a_6$ (`[E11]`) naively requires evaluating $\Sigma_0(a_9)$ and a_6 across those candidates. We now show how **Carry-Sudoku** (see Companion Guide §4) eliminates 99.999% **of all** e_{13} **branches** using causal lower-triangular bit-slices *before* $\Sigma_0(a_9)$ or a_6 is ever computed.

5.1 The Exact Equivalence Theorem (`[E23]`)

Theorem 1 (Carry-Sudoku Exact Equivalence). *Under* `[E9]`, *define* $X := \texttt{0xffffffff} - a_9 + \Sigma_0(a_8)$, $D := a_8 \oplus a_7$, *and* $S := X + \mathrm{Ch}(D, X, a_8) \pmod{2^{32}}$. *Then:*

$$a_5 = a_6 \iff \Big(\big((S \oplus X) \wedge D\big) = 0\Big) \ \wedge \ \Big(\big((a_6 \oplus S) \wedge \neg D\big) = 0\Big). \tag{E23}$$

Proof. By `[E9]`, $a_5 = X + \mathrm{Maj}(a_8, a_7, a_6)$. Decompose $\mathrm{Maj}(a_8, a_7, a_6)$ bitwise with respect to $D = a_8 \oplus a_7$:

- On bit positions where $D_b = 0$ ($a_{8,b} = a_{7,b}$), $\mathrm{Maj}(a_8, a_7, a_6)_b = a_{8,b}$ is independent of a_6.
- On bit positions where $D_b = 1$ ($a_{8,b} \neq a_{7,b}$), $\mathrm{Maj}(a_8, a_7, a_6)_b = a_{6,b}$.

Hence $\mathrm{Maj}(a_8, a_7, a_6) = (D \wedge a_6) \oplus (\neg D \wedge a_8) = \mathrm{Ch}(D, a_6, a_8)$. Now compare $a_5 = X + \mathrm{Ch}(D, a_6, a_8)$ with $S = X + \mathrm{Ch}(D, X, a_8)$ by induction from bit $b = 0$ to 31:

- Suppose $a_5 = a_6$ on bits $0..b-1$, so the incoming carry c_b into bit b of $X + \mathrm{Ch}(D, a_6, a_8)$ equals the incoming carry into bit b of $S = X + \mathrm{Ch}(D, X, a_8)$:
- If $D_b = 1$, then $\mathrm{Ch}(D, a_6, a_8)_b = a_{6,b}$, so $a_{5,b} = X_b \oplus a_{6,b} \oplus c_b$. Thus $a_{5,b} = a_{6,b} \iff X_b \oplus c_b = 0$, and when $X_b \oplus c_b = 0$, the outgoing carry from bit b is $\mathrm{Maj}(X_b, a_{6,b}, c_b) = X_b = \mathrm{Maj}(X_b, X_b, c_b)$, which is **identical** to the outgoing carry of $S = X + \mathrm{Ch}(D, X, a_8)$ where $S_b = X_b \oplus X_b \oplus c_b = c_b = X_b$, i.e. $(S_b \oplus X_b) = 0$!
- If $D_b = 0$, then $\mathrm{Ch}(D, a_6, a_8)_b = a_{8,b} = \mathrm{Ch}(D, X, a_8)_b$, so $a_{5,b} = S_b$ and the outgoing carry is again identical to that of S. Thus $a_{5,b} = a_{6,b} \iff a_{6,b} = S_b$. □

5.2 7-Stage Causal Bit-Slice Pruning & Rotation-Invariant $\Sigma_0(a_9)$ Windows (`[E24]` – `[E26]`)

Theorem 1 has a remarkable algorithmic consequence: the first condition in `[E23]`,

$$\big((S \oplus X) \wedge D\big) = 0, \qquad S = X + \mathrm{Ch}(D, X, a_8), \tag{E24}$$

does not depend on a_6 **or** $\Sigma_0(a_9)$ **at all**, and because $a_9 = e_{13} - c_{e_{13}}$, $X = c_{a_5} - a_9$, and $a_7 = c_{a_7,\text{full}} - (e_{14} \wedge e_{13}) + \big((a_{10} \oplus a_8) \wedge a_9\big)$ involve only bitwise operations and modular additions/subtractions, **the lowest** k **bits of** $X, a_7, D,$ **and** S **depend exclusively on the lowest** k **bits of** e_{13}!

Just like eliminating invalid numbers cell-by-cell in Sudoku before filling the rest of the grid, `ctx_gen_worker` (CPU) and `ctx_gen_cuda_kernel` (GPU) assign e_{13} from LSB to MSB in **7 causal stages**:

$$\begin{aligned}
\text{Stage 1a (bits 0..3):} \quad & \big((S \oplus X) \wedge D \wedge \texttt{0xf}\big) = 0, \\
\text{Stage 1b (bits 4..7):} \quad & \big((S \oplus X) \wedge D \wedge \texttt{0xff}\big) = 0 \implies \text{fixes } R_{\text{lo}} \text{ and } (e_{13})_{18} \oplus (e_{13})_{13}, \\
\text{Stage 1c (bits 8..11):} \quad & \big((S \oplus X) \wedge D \wedge \texttt{0xfff}\big) = 0, \\
\text{Stage 1d (bits 12..14 + exact bit 18):} \quad & \big((S \oplus X) \wedge D \wedge \texttt{0x7fff}\big) = 0, \\
\text{Stage 1e (bits 15..17, checked to bit 18):} \quad & \big((S \oplus X) \wedge D \wedge \texttt{0x7ffff}\big) = 0 \implies \text{fixes bits 25..31 via [E21]}.
\end{aligned} \tag{E25}$$

Notice that in Stage 1d, once bit 13 of e_{13} is chosen, bit 18 of e_{13} is immediately determined by $(R_{\text{lo}} \gg 7) \oplus (e_{13})_0 \oplus (e_{13})_{13}$ (`[E20]`), allowing Stage 1e to test a **19-bit Carry-Sudoku mask (** `0x7ffff` **)** on 18 chosen bits *before* $\Sigma_1(e_{13,\text{base}})$ is even computed!

Next, once bits $0..18$ and $25..31$ are fixed (26 of 32 bits of e_{13}), only bits $19..24$ (`s19` and `s22`) remain free. Under $\Sigma_0(a_9) = \mathrm{ROR}(a_9,2) \oplus \mathrm{ROR}(a_9,13) \oplus \mathrm{ROR}(a_9,22)$, bits $19..24$ of a_9 (allowing 1 carry bit into bit 25) land exclusively in bit ranges `[17..23]` , `[6..12]` , and `[29..31, 0..2]` . Consequently, **bits `13..15` and `3..4` of $\Sigma_0(a_9)$ are 100% rotation-invariant across all 64 values of bits $19..24$!** Since S and D are already fixed through bit 18, we check the exact equivalence `a6_0 == a5_0` on those two rotation-invariant slices (`Stage 2`) before looping over `s19` (`Stage 3a` , mask `0x3fffff`) and `s22` (`Stage 3b` , full 32-bit `[E23]`):

$$\big(((a_{6,0} - a_{5,0}) \gg 13) + 1\big) \wedge \texttt{0xe} = 0 \quad \wedge \quad \big(((a_{6,0} - a_{5,0}) \gg 3) + 1\big) \wedge \texttt{0x6} = 0. \tag{E26}$$

On GPU (`ctx_gen_cuda_kernel`), we map the Causal Carry-Sudoku tree onto warp-cooperative lockstep stages using `__ballot_sync` and `__shfl_sync` , achieving $29{,}310$ **exact $D = 0$ padded contexts per second** on 1× NVIDIA L4 GPU (`56 trillion` effective e_{13} trials/s).

---

6. Slot-Sudoku & Zero-Table Closed-Form $\mathbb{F}_2$ Inverse for Left-to-Right Q_0^{-1} Chains

With an exact $D = a_5 \oplus a_6 = 0$ padded Right context in hand, we sweep $a_1 \in [0, 2^{32})$ and solve the three chained Q_0^{-1} equations `[E14]` from Left to Right.

6.1 Closed-Form 5-Instruction $\mathbb{F}_2$ Inverse (`reconstruct_u` , `[E27]` – `[E29]`)

Storing full 32-bit preimages u of $y = Q_0(u) = \sigma_0(u) - u$ requires 4 bytes per slot. Splitting $u = (u_{\text{hi}} \ll 16) \mid u_{\text{lo}}$ with $u_{\text{lo}}, u_{\text{hi}} \in [0, 65535]$, modular addition gives $(u + y) \bmod 2^{17} = \sigma_0(u) \bmod 2^{17}$, and $\mathbb{F}_2$-linearity of σ_0 gives:

$$\sigma_0(u_{\text{hi}} \ll 16) \equiv (u + y) \oplus \sigma_0(u_{\text{lo}}) \pmod{2^{17}}. \tag{E27}$$

Previously (`[E25]` in earlier drafts), we inverted `[E27]` using a 128 KB lookup table `REC_HI16[65536]` . However, on GPUs (and CPU L1 caches), issuing 6–8 random 16-bit loads per `(a1, ctx)` into a 128 KB table (~200 billion random loads per sweep) pollutes L1 data cache and contends with Q_0^{-1} table loads.

We now prove that `[E27]` has an **exact 5-instruction register closed form** with zero memory lookups:

Theorem 2 (Closed-Form $\mathbb{F}_2$ Inverse of Upper 16 Bits). *Let $v = (u_{\text{lo}} + y) \oplus \sigma_0(u_{\text{lo}}) \pmod{2^{32}}$, and let $U = u_{\text{hi}} \ll 16 \in \mathbb{F}_2^{32}$ have bits in positions $16..31$. Define the linear operator $L(T) := T \oplus (T \ll 11) \oplus (T \ll 15) \pmod{2^{32}}$ on upper-half words T. Then L is an involution-like triangular bijection on bits $16..31$ with $U = L(T)$, where T is given in closed form by:*

$$\begin{aligned} b_0 &= \big((v \ll 15) \oplus (v \ll 11) \oplus v\big) \wedge \texttt{0x00010000}, \\ b_{12} &= \big((v \ll 14) \oplus (v \ll 3)\big) \wedge \texttt{0x00060000}, \\ T &= b_0 \mid b_{12} \mid \big((v \ll 18) \wedge \texttt{0xfff80000}\big), \end{aligned} \tag{E28}$$

and the full 32-bit preimage is recovered in 5 bitwise instructions (`reconstruct_u_dev`):

$$u = u_{\text{lo}} \mid \big(T \oplus (T \ll 11) \oplus (T \ll 15)\big). \tag{E29}$$

Proof. On bits $0..16$, $\sigma_0(U) = \mathrm{ROR}(U,7) \oplus \mathrm{ROR}(U,18) \oplus (U \gg 3)$ places $U_{16..31}$ at bits $9..24$, bits $30..31 \cup 0..13$, and bits $13..28$. Defining $T = L^{-1}(U)$ eliminates the two shift terms $(T \ll 11)$ and $(T \ll 15)$ inside $\mathrm{ROR}(U,18)$, leaving a single 3-band triangular relationship between $T_{16..31}$ and $v_{0..16}$, which resolves directly into bit 16 (b_0), bits $17..18$ (b_{12}), and bits $19..31$ ($v \ll 18$). Verified exhaustively across all $2^{16} = 65{,}536$ states with 0 errors. □

6.2 Slot-Sudoku: 6 GB 2-Tier Nested 4-Slot EDF Table & 16 GB / 32 GB Direct-Indexed Tables (`[E30]` – `[E33]`)

Because $Q_0(u) = \sigma_0(u) - u$ behaves as a random mapping on $\mathbb{Z}_{2^{32}}$, the number of preimages $|Q_0^{-1}(y)|$ follows a $\mathrm{Poisson}(1)$ distribution (36.79% of y have 0 roots, 36.79% have 1, 18.39% have 2, 6.13% have 3, 1.53% have 4, and 0.37% have ≥ 5).

We design three memory-tier tables using **Slot-Sudoku** (see Companion Guide §5):

1. **6.00 GB 2-Tier Nested 4-Slot EDF + Cuckoo-Sudoku Table (`/tmp/q0_u16_sudoku6.bin` , `solve20_padded_fast.c`):** Designed to fit comfortably inside a 16 GB Mac M3 or 24 GB NVIDIA L4 GPU (3×2^{30} entries of `uint16_t` = `6.00 GB`), where every 16 consecutive y values (a **quad** $q = y \gg 4$) share **12 contiguous 16-bit slots**:

- **Earliest-Deadline-First (EDF) 3-Slot Sliding Window (`[E30]`):** At Levels 1 and 2, each y probes a 3-slot window within its 12-slot quad at offset:

$$\text{slide3_off}(y) = (y \gg 1) + (y \gg 2) - \big(\text{ROR}(\texttt{0x6AAA6AAA}, y \wedge 31) \gg 31\big). \tag{E30}$$

- **Cuckoo-Sudoku Eviction & Nested 4th Slot (`[E31]`):** During table construction, after 4 hierarchical overflow passes and EDF ordering, a **Cuckoo-Sudoku solver** (`sudoku_upgrade_worker_6gb` and `tier2_nest4_quad_worker`) searches alternating relocation paths within each 12-slot quad to pack displaced preimages (`+48.66M` Tier-1 and `+36.32M` Tier-2 preimages), placing any 4th root of y at the deterministic adjacent slot $\text{b}[(y \wedge 15) \geq 13 \ ? \ -1 : 3]$ (probed **only** when all 3 primary slots are valid roots of y, adding 0 overhead otherwise!). At Level 3, all 12 slots of the quad (`3 × uint64_t`) are tested simultaneously via 64-bit SWAR (`[E35]`):

$$6.00\text{ GB Coverage} = \frac{3{,}563{,}795{,}213}{2^{32}} = 82.976\%, \quad \text{3-Level Survival} = (0.82976)^3 = 57.13\%. \tag{E31}$$

2. **16.00 GB Direct-Indexed 2-Way Exact GPU VRAM Table (`d_H_direct2[y]` , `solve20_padded_cuda.cu`):** Built natively in GPU VRAM in **`3.13 seconds`** , storing up to two exact 16-bit roots per y in a single `uint32_t` (`d_H_direct2[y]`):

$$16.00\text{ GB Exact Coverage} = 79.64\%, \quad \text{3-Level Survival} = (0.7964)^3 = 50.51\%. \tag{E32}$$

   Because `d_H_direct2[y]` is direct-indexed by exact 32-bit y, every non-zero `ulo` in `d_H_direct2[y]` is guaranteed to satisfy $Q_0(u) = y$ (`100%` exact), eliminating all false-positive reconstructions and `Qf(u) == y` checks across Levels 1, 2, and 3!

3. **32.00 GB Direct-Indexed 4-Way Exact CPU Table (`H_exact32[y]` , **`--table-gb 32`**):** Stores up to four 16-bit roots per y in one `uint64_t` :

$$32.00\text{ GB Exact Coverage} = \frac{4{,}276{,}521{,}180}{2^{32}} = 99.5705\%, \quad \text{3-Level Survival} = 98.717\%. \tag{E33}$$

6.3 The 3-Level Tree Search, 64-Bit SWAR Level-3 Filter, and Terminal 20-Step Bijection (`[E34]` – `[E37]`)

For each $a_1 \in [0, 2^{32})$ and each Right context in a batch of size B:

- **Level 1 ($a_1 \to W_1 \to a_2$) & Level 2 ($a_2 \to W_2 \to a_3$) with Early Hamming Pruning (`[E34]`):** From a_1, look up $W_1 \in Q_0^{-1}(T_{16} + F_1(a_1))$, set $a_2 = W_1 + G_1(a_1)$, look up $W_2 \in Q_0^{-1}(T_{17} + G_2(a_1, a_2) - W_1)$, and set $a_3 = W_2 + G_2$. Before computing e_3, G_3 or touching Level-3 memory, we prune any a_3 whose majority mask $M_3 = \neg(a_3 \oplus a_5)$ has Hamming weight exceeding $w_{\text{thresh}} \in \{13, 14\}$:

$$\text{popcount}\big(\neg(a_3 \oplus a_5)\big) \leq w_{\text{thresh}}, \tag{E34}$$

   eliminating 81.8% **($w_{\text{thresh}} = 13$) to** 71.2% **($w_{\text{thresh}} = 14$)** of Level-3 DRAM lookups before execution.

- **Level 3: 64-Bit SWAR Pre-Reconstruction Filter & 19-Step Padded Solution Stream (`[E35]`):** Because the majority condition $((a_4 \oplus a_5) \wedge M_3) = 0$ (`[E13]`) is lower-triangular on bits $0..14$, any candidate u_{lo} must satisfy $(((u_{\text{lo}} + G_3) \oplus a_5) \wedge M_3 \wedge \texttt{0x7fff}) = 0$ **before** u_{hi} is reconstructed! In the 6 GB table, we test 4 slots at a time in a single 64-bit word `w` using SWAR carry-bit detection (`K7 = 0x7fff7fff7fff7fff` , `K8 = 0x8000800080008000` , `K1 = 0x0001000100010001`):

$$d = \Big(\big(((w \wedge K_7) + g_{64}) \oplus A_{64}\big) \wedge m_{64}\Big) + K_7, \qquad \text{4-slot hit mask} = \neg d \wedge K_8, \tag{E35}$$

   rejecting $> 99\%$ of 12-slot quads in 3 register operations without unpacking a single slot! Every candidate passing the full 32-bit check $((a_4 \oplus a_5) \wedge M_3) = 0$ with $Q_0(W_3) = t_{18}$ is a **complete, verified 19-step padded preimage** (`local_sol19++`).

- **Terminal 20-Step Filter & Exact $\mathbb{F}_2^{32}$ Bijection (`[E36]` – `[E37]`):** For each 19-step padded solution (`local_sol19`), we recover W_4 via `[E7]` and test the step-19 schedule equation:

$$\sigma_0(W_4) + W_3 \equiv T_{19} \pmod{2^{32}}, \qquad T_{19} := W_{19} - \sigma_1(W_{17}) - W_{12}. \tag{E36}$$

  Furthermore, viewing $\sigma_0(x) = \mathrm{ROR}(x,7) \oplus \mathrm{ROR}(x,18) \oplus (x \gg 3)$ as a 32×32 matrix over $\mathbb{F}_2$, we prove that $\det(\sigma_0) = 1 \pmod 2$ (`[E37]`):

$$\det_{\mathbb{F}_2}(\sigma_0) = 1 \implies W_4 = \sigma_0^{-1}\big((T_{19} - W_3) \bmod 2^{32}\big) \text{ is uniquely determined by } W_3 \text{ and } T_{19}! \tag{E37}$$

6.4 Multi-Context T_{16} Coalescing, Multi-Way Single-T_{16} Fusion & Warp-Compacted L3 Queues

To maximize memory bandwidth on both Apple M3 and NVIDIA L4 GPUs:

1. **Same-Sector T_{16} Pairing ($\Delta T_{16} \le 16$):** By maintaining a sorted reservoir of $D = 0$ contexts ordered by T_{16}, adjacent pairs satisfy $\Delta T_{16} = T_{16,1} - T_{16,0} \le 16$, so their Level-1 lookups $t_{16,0} = T_{16,0} + F_1(a_1)$ and $t_{16,1} = T_{16,1} + F_1(a_1)$ fall into the **exact same 64-byte CPU cache line / 32-byte GDDR6 sector** across all 2^{32} values of a_1.
2. **Multi-Way Single-T_{16} Fusion ($B = 16$, `Idea 5`) & Warp-Compacted Shared-Memory Level-3 Queue (`Idea 7`):** Because Level 1 (`W1, a2, e2, sm2, G2`) depends on the Right context *only* through the 32-bit scalar T_{16}, evaluating $B = 16$ contexts sharing the same T_{16} offset (or evaluating 2^{16} right contexts sharing T_{16}, T_{17}, T_{18} simultaneously via `[E37]`) amortizes Level 1 and the `(a2, e2, sm2, G2)` bridge $16\times$, while compacting surviving Level-2 tasks into a warp-shared queue (`__ballot_sync`) eliminates SIMD divergence at Level 3, reaching $180{,}831.9$ sol19/s on 1× NVIDIA L4 GPU!

---

7. Companion Result: 20-Step Unpadded Compression Preimages via the Zero-a_3 Bridge

When FIPS 180-4 padding is not required (the 20-step compression-function model studied by Basra), we can eliminate the $(3/4)^{32} \approx 2^{-13.28}$ majority survival loss of `[E13]` altogether by placing the state on the **Zero-a_3 Manifold** (`solve20_zero_a3.c`):

$$a_3 = 0, \qquad a_5 = 0, \qquad a_6 = 0, \qquad a_8 = 0, \qquad a_9 = 0, \qquad a_{10} = \texttt{0xffffffff}. \tag{E38}$$

Because $\mathrm{Maj}(0, x, 0) = 0$, both majority functions containing a_4 vanish identically for 100% **of** $a_4 \in [0, 2^{32})$:

$$\mathrm{Maj}(a_5, a_4, a_3) = \mathrm{Maj}(0, a_4, 0) = 0, \qquad \mathrm{Maj}(a_6, a_5, a_4) = \mathrm{Maj}(0, 0, a_4) = 0. \tag{E39}$$

Fixing $a_3 = 0$ removes one degree of freedom on the Left, which we restore on the Right by leaving a_7 free and solving the step-16 schedule equation via the companion operator $Q_1(u) = \sigma_1(u) + u$:

$$Q_0(W_1) = F_{\text{left}}(a_1) + R_{16}(a_7, a_{11}, a_{12}), \qquad Q_1(\Delta_{14}) = \mathrm{Target}_{Q_1}(a_1, a_{11}, a_{12}). \tag{E40}$$

Precomputing 16 GB tables for Q_0^{-1}, Q_1^{-1}, and the Left bridge (a_1, W_1, W_2) satisfying $a_3 = 0$ (48 GB total, 100% independent of H^*), `solve20_zero_a3.c` achieves **zero β-filter loss** (63.37% survival per bridge hit) and **2.16 million exact 19-step compression solutions per second** on a 16-vCPU CPU node:

$$\text{Total 20-Step Unpadded Compression Solve Time} = 1{,}266.26 \text{ s } (21.1 \text{ minutes on 16 CPU cores}). \tag{E41}$$

---

8. Verified 19-Step and 20-Step Padded SHA-256 Preimage Witnesses

All three witnesses below are independently verified by running `python3 verify_all_witnesses.py` (`verify_all_witnesses.py`).

8.1 Witness 1: First Demonstrated 20-Step Padded SHA-256 Preimage ($H^*_{20} = 0000\ldots0000$)

Found by `solve20_padded_cuda.cu` on Sweep 62 (`ctx=5`, after $70{,}661{,}112$ exact 19-step padded solutions; recorded in `FOUND_20ROUND_PREIMAGE.txt`):

- **Target 20-Step Digest (H^*_{20}):** `0000000000000000000000000000000000000000000000000000000000000000`
- **55-Byte Message M (440 bits, big-endian hex):** `ec531306c85ad4a4ad211d2e1a03ece4d0ad4a0e6b4c2b9436b3babdcb-d520597196f52c042b8cd95f1a39f71b58b47b0647c78e65416f`
- **FIPS 180-4 Padded 512-Bit Message Block ($W_0 \ldots W_{15}$):**

```
W[ 0.. 3] = ec531306  c85ad4a4  ad211d2e  1a03ece4
W[ 4.. 7] = d0ad4a0e  6b4c2b94  36b3babd  cbd52059
W[ 8..11] = 7196f52c  042b8cd9  5f1a39f7  1b58b47b
W[12..15] = 0647c78e  65416f80  00000000  000001b8
```

  (Note the mandatory FIPS 180-4 padding: $W_{13} \bmod 256 = \texttt{0x80}$, $W_{14} = \texttt{0x00000000}$, and $W_{15} = \texttt{0x000001b8} = 440$ bits.)

Complete 20-Step Internal State Trajectory (a_t, e_t, W_{t-1}) of the 20-Step Padded Witness

Step t	Register a_t	Register e_t	Message / Schedule Word W_{t-1}	Algebraic / Carry-Sudoku Invariant Verified
1	`e85b9b53`	`851af5a8`	$W_0 =$ `ec531306`	Swept root $a_1 \in [0, 2^{32})$
2	`47d401c6`	`4b125b48`	$W_1 =$ `c85ad4a4`	Level-1 Q_0^{-1} hit: $Q_0(W_1) = T_{16} + F_1(a_1)$
3	`0a280ce0`	`8b761050`	$W_2 =$ `ad211d2e`	Level-2 Q_0^{-1} hit; $\text{popcount}(M_3) = 10 \leq 14$
4	`67298bc8`	`c121961d`	$W_3 =$ `1a03ece4`	Level-3 Q_0^{-1} hit; $((a_4 \oplus a_5) \wedge M_3) = 0$ (`[E13]`)
5	**`67978bcd`**	`2724a33e`	$W_4 =$ `d0ad4a0e`	**Exact Shaping (`[E11]`):** $a_5 =$ `67978bcd` ($D = 0$)
6	**`67978bcd`**	`eea015fa`	$W_5 =$ `6b4c2b94`	**Exact Shaping (`[E11]`):** $a_6 = a_5 =$ `67978bcd` ($D = 0$)
7	`41b1f1b0`	`8aa086f2`	$W_6 =$ `36b3babd`	Closed-form a_7 absorbing $W_{14} = 0$ (`[E17]`)
8	`edb02749`	`9319765f`	$W_7 =$ `cbd52059`	Invariant a_8 absorbing $W_{15} = 440$ (`[E17]` , `[E22]`)
9	`7fd85be0`	**`ffffffff`**	$W_8 =$ `7196f52c`	**Selector Saturation (`[E8]`):** $e_9 =$ `0xffffffff`
10	**`a7b351d7`**	**`ffffffff`**	$W_9 =$ `042b8cd9`	**Selector Saturation (`[E8]`):** $e_{10} =$ `0xffffffff` , $a_{10} = a_{11}$
11	**`a7b351d7`**	`4f9e3855`	$W_{10} =$ `5f1a39f7`	**Majority Simplification (`[E22]`):** $a_{11} = a_{10} =$ `a7b351d7`
12	`e5d45365`	`81ba7166`	$W_{11} =$ `1b58b47b`	Right context seed a_{12}
13	`bf9f516c`	`e4acad89`	$W_{12} =$ `0647c78e`	Target-fixed a_{13}; e_{13} solves `[E21]` & `[E23]`
14	`b4322266`	`da868879`	$W_{13} =$ `65416f80`	**Padding byte (`[E21]`):** $W_{13} \bmod 256 =$ `0x80`
15	`8493b564`	**`ffffffff`**	$W_{14} =$ `00000000`	Ch-**Annihilation (`[E22]`):** $e_{15} =$ `0xffffffff`, $W_{14} = 0$
16	`97457a44`	`03b1406f`	$W_{15} =$ `000001b8`	**Padding length (`[E1]`):** $W_{15} = 440$ bits (`0x1b8`)
17..20	`5ab00ac6` .. `95f61999`	`a41f32e7` .. `aef1ad81`	$W_{16..19} =$ `d6317d0a` .. `cd1c5aee`	Terminal hit $\sigma_0(W_4) + W_3 = T_{19} \implies H_{20} =$ `00..00`!

8.2 Witness 2: Demonstrated 19-Step Padded SHA-256 Preimage ($H_{19}^* = \text{SHA-256}(\text{“abc”})$)

- **Target 19-Step Digest ($H_{19}^* = \text{SHA-256}(\text{“abc”})$):** `ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad`
- **55-Byte Message M (440 bits, big-endian hex):** `8ca254f8d04dd3fb04efaad6b1867588ffe833de49a144fedbf1839422892dc9f5a9158d8fc063290a14a9a6c55dfa245642a0247ec45a`
- **FIPS 180-4 Padded 512-Bit Message Block ($W_0 \ldots W_{15}$):**

```
W[ 0.. 7] = 8ca254f8 d04dd3fb 04efaad6 b1867588 ffe833de 49a144fe dbf18394 22892dc9
W[ 8..15] = f5a9158d 8fc06329 0a14a9a6 c55dfa24 5642a024 7ec45a80 00000000 000001b8
```

8.3 Witness 3: Demonstrated 20-Step Unpadded Compression Preimage (Zero-a_3 Bridge)

- **Target 20-Step Digest (H_{20}^*):** `deadbeefcafebabe0123456789abcdeffedcba987654321013579bdf2468ace0`
- **512-Bit Message Block ($W_0 \ldots W_{15}$, solved in 21.1 min on 16 CPU cores via `solve20_zero_a3.c`):**

```
W[ 0.. 7] = 4334d956 6eccece1 6727a1b2 d85811af 890c5399 79018a3b d346b94d 5f26009b
W[ 8..15] = 7562d311 0bfd80d5 1f8c38f4 16d8315d e0353aa2 6adf4bd2 a2bf14a2 801ae8f7
```

---

9. Experimental Benchmarks & Related Work Comparison

9.1 End-to-End Progression Across CPU (Apple M3) and GPU (NVIDIA L4)

Solver Stage / Architecture	Table Size & Structure	Right-Context Speed ($D = 0$)	19-Step Padded Rate (`sol19/s`)	Expected 20-Step Padded Time (1 Device)	Expected Time on 8× L4 GPUs
v1 `src/pre19.c` (CPU, 12 threads)	2.0 GB CSR (2^{29} slots)	Soft shaping ($D \leq 3$)	~ `20` – `400` `sol19/s`	~ `7.6+ days` (with soft loss)	—
Carry/Slot-Sudoku CPU (`solve20_padded_fast.c` , 12 M3 threads)	**6.0 GB 2-Tier Nested 4-Slot Sudoku**	**`86.4B` `trials/s` (`0.11s` /ctx)**	**`6,242.8` `sol19/s`** (`1,609` ms/ctx)	**`11.9 hours`** ($15.3\times$ faster!)	—
Cycle-0 CUDA Baseline (1× L4 GPU)	6.0 GB 2-Tier Sudoku ($B = 2, \Delta T_{16} > 0$)	`8,370 ctx/s`	`53,847.5` `sol19/s`	`1.38 hours`	`10.4 min`
Upgraded 6 GB CUDA (`solve20_padded_cuda.cu` , 1× L4 GPU)	**6.0 GB + Zero-Table `[E29]` + Same-Sector $B = 8$**	`7,580 ctx/s`	**`73,560.5` `sol19/s`** (`70.0k` net)	**`1.06 hours`**	**`8.0 min`**
Cycle-1 CUDA (`16 GB Direct 2-Way` , 1× L4 GPU)	**16.0 GB Direct 2-Way Exact + Exact-Twin $B = 2$**	`8,370 ctx/s`	**`132,888.7` `sol19/s`** (`2.47×`)	**`33.7 min`**	**`4.2 min`**
Cycle-2 CUDA (`16 GB Direct + Multi-T16 + Warp-L3` , 1× L4)	**16.0 GB Direct + 16-Way T_{16} Fusion + Warp L3**	**`29,310.3` `ctx/s`** (`3.50×`)	**`180,831.9` `sol19/s`** (`3.36×`)	**`24.7 min`**	**`3.1 min`**

9.2 Comparison with Prior Reduced-Step SHA-256 Preimage Results

Work	Steps	Attacked Object	FIPS 180-4 Padding?	Target Type	Cost / Runtime	Demonstrated Witness?
Isobe–Shibutani (FSE 2009)	24	Hash (1 block)	Yes	Arbitrary	2^{240} calls	No (theoretical)
Aoki et al. (ASIACRYPT 2009)	43	Hash	Yes	Arbitrary	$2^{254.9}$ calls	No (theoretical)
Khovratovich–Rechberger–Savelieva (FSE 2012)	45 / 52	Hash / Compression	Yes / No	Arbitrary	$2^{255.5}/2^{255}$	No (theoretical)
Guo et al. (EUROCRYPT 2026)	46 / 47	Reduced SHA-256	Yes	Arbitrary	$2^{254.5}/2^{255.6}$	No (theoretical)
Homsirikamol et al. (CISIM 2012, ePrint 2012/421)	16	Hash (1 block, 446-bit)	Yes	Fixed	~192 core-h	No (not printed in paper)
McDair (metzdowd list, Mar 2024)	17	Hash (31B / 55B)	Yes	1 challenge (`de-b360ae...`)	Unspecified (no method)	Yes (3 messages verified at 17 steps)
Davydov et al. (2025)	17 / 18	Compression	No	All-zero (`00..00`)	Seconds (SAT)	Yes (unpadded only)
Zaikin (*Constraints*, Jan 2026)	19	Compression	No	All-ones (`ff..ff`)	18.5 h on 192 cores	Yes (unpadded only)
Basra (GitHub manuscript, Sep 2026)	19 / 20	Compression	No	Arbitrary / `ff..ff`	~0.01 s (19) / 2.45–16.2 GPU-h (20)	Yes (unpadded only)
This Work (`solve20_zero_a3.c`)	**20**	**Compression**	**No**	`deadbeef...`	**21.1 min on 16 CPU cores**	**Yes (§8.3, `WITNESS_20ROUND.md`)**
This Work (`pre19.c` / `solve20_padded_-fast.c`)	**19**	**Full Hash (55B / 119B)**	**Yes (FIPS 180-4)**	**Public-label digests**	**< 1 ms on M3 / GPU** (18.0 s median v1)	**Yes (37+ witnesses, §8.2)**
This Work (`solve20_-padded_cuda.cu` / `solve20_padded_-fast.c`)	**20**	**Full Hash (55B)**	**Yes (FIPS 180-4)**	**`0000...0000` / Arbitrary**	**~19 min actual (3.1–8.0 min upgraded 8× L4; 11.9 h 1× M3)**	**Yes (§8.1, `FOUND_20ROUND_PREIMAGE.txt`)**

Disclosure & Attribution

The selector-saturation and context-shaping principle ($e_9 = e_{10} = \texttt{0xffffffff}$, $a_5 \approx a_6$) is due to K. S. Basra (`https://github.com/kamb-code/sha256-r19-preimage`), whose manuscripts note substantial assistance from AI tools. Both `srw/sha256-19step-preimages-with-padding` and the Carry-Sudoku / Slot-Sudoku solvers and witnesses in this work were developed with AI assistance.

References

1. **FIPS 180-4:** National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015.
2. **Isobe, T., Shibutani, K. (FSE 2009):** *Preimage Attacks on Reduced Tiger and SHA-2*, Fast Software Encryption 2009, LNCS 5665, pp. 139–155.
3. **Aoki, K., Guo, J., Matusiewicz, K., Sasaki, Y., Wang, L. (ASIACRYPT 2009):** *Preimages for Step-Reduced SHA-2*, ASIACRYPT 2009, LNCS 5912, pp. 578–597.
4. **Khovratovich, D., Rechberger, C., Savelieva, A. (FSE 2012):** *Bicliques for Preimages: Attacks on Skein-512 and the SHA-2 Family*, FSE 2012, LNCS 7549, pp. 244–263.

5. **Homsirikamol, E., Morawiecki, P., Rogawski, M., Srebrny, M. (CISIM 2012):** *Security Margin Evaluation of SHA-256 through SAT-Based Cryptanalysis*, CISIM 2012, IACR ePrint 2012/421.
6. **Davydov, A., Pikhtovnikov, A., Kiryanova, E., Zaikin, O. (2025):** *Cryptographic Strength Analysis of the SHA-256 Hash Function Using the SAT Approach*, Scientific and Technical Journal of Information Technologies, Mechanics and Optics, 2025.
7. **Zaikin, O. (2026):** *Inverting 19-Round SHA-256 Compression Functions via Cube-and-Conquer*, Constraints, Jan 2026.
8. **Guo, J., Li, M., Liu, G., Wang, M., Zhang, L. (EUROCRYPT 2026):** *Improved Meet-in-the-Middle Preimage Attacks on Step-Reduced SHA-256/512*, EUROCRYPT 2026, IACR ePrint 2026/353.
9. **Basra, K. S. (2026):** *Context Shaping for 19- and 20-Round SHA-256 Compression Preimages*, GitHub manuscript (`https://github.com/kamb-code/sha256-r19-preimage`), Sep 2026.
10. **Wain, S. (2026):** *Carry-Sudoku & Slot-Sudoku in Reduced SHA-256: A Self-Contained ELI5 and ELI-Undergraduate Companion Guide*, Companion Document `COMPANION_SUDOKU_ELI5_ELIU.pdf` , Oct 2026.